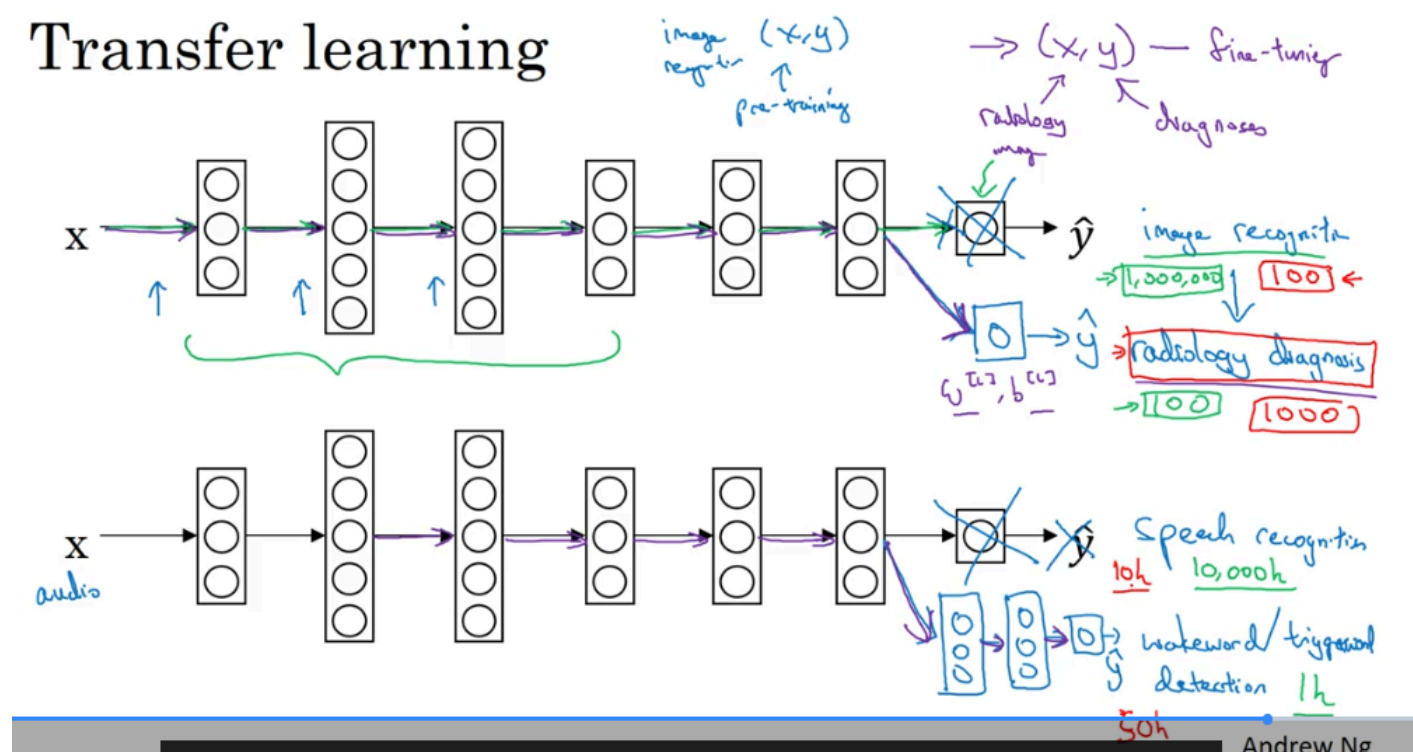


Learning from Multiple tasks

Transfer Learning

One of the most powerful ideas in deep learning is that sometimes you can take knowledge the neural network has learned from one task and apply that knowledge to a separate task. So for example, maybe you could have the neural network learn to recognize objects like cats and dogs and then use that knowledge or use part of that knowledge to help you do a better job reading x-ray scans. This is called transfer learning. Let's take a look.



Transfer Learning (Main Idea)

- Transfer learning = **reuse knowledge from one task to help another task**
- Example:
 - Train model to recognize **cats, dogs, objects**
 - Then use it to analyze **X-ray images**

How It Works

1. **Train on first task (pre-training)**
 - Input: normal images
 - Output: object labels (cat, dog, etc.)
 - Model learns general features (edges, shapes, patterns)
2. **Modify the model**
 - Remove the **last layer**
 - Add a new output layer for the new task (e.g., X-ray diagnosis)
3. **Train on new task**
 - Input: X-ray images
 - Output: diagnosis

Training Options

- **Small dataset**
 - Only train the **last layer**
 - Keep other layers fixed
- **Large dataset**

- Train **all layers** (fine-tuning)
-

Important Terms

- **Pre-training:** training on the first task (e.g., image recognition)
 - **Fine-tuning:** training on the new task (e.g., radiology)
-

Why It Works

- Early layers learn **basic features**:
 - edges
 - curves
 - shapes
 - These features are useful for many tasks → helps:
 - learn faster
 - need less data
-

Takeaway

- Transfer learning = **smart reuse of knowledge**
- Very useful when you **don't have much data** for the new task

Another example → Transfer Learning (Speech Example)

- You train a **speech recognition model**:
 - Input: audio
 - Output: text (transcript)
 - Now you want to build a **wake word detector**:
 - Examples: "Hey Siri", "OK Google", "Alexa"
-

How to Apply Transfer Learning

- Remove the **last layer** of the model
 - Add:
 - A new output layer **or**
 - Several new layers (for better learning)
 - Then train on wake word data
-

Training Options

- **Little data**
 - Train only the **new layers**
 - **More data**
 - Train more layers (or entire network)
-

When Transfer Learning Works Well

- When:
 - **Source task has a lot of data**
 - **Target task has little data**

Examples:

- Image recognition (1M images) → Radiology (100 images)
- Speech recognition (10,000 hours) → Wake word (1 hour)

👉 The model already learned:

- patterns of images
- structure of human speech

→ So it helps learning faster with less data

When It Does NOT Help Much

- When target task already has **enough or more data**

Examples:

- Only 100 images for source, but 1000 radiology images
- 10 hours speech data, but 50 hours wake word data

👉 In this case:

- Target data is **more valuable**
- Transfer learning gives **little or no improvement**

Key Idea

- Transfer learning works best:
→ **Big data** → **Small data**

Takeaway

- Use transfer learning when:
 - You have **a lot of data in one task**
 - But **very little data in your actual task**
- Don't rely on it when:
 - Your target task already has **plenty of data**

When transfer learning makes sense

When transfer learning makes sense

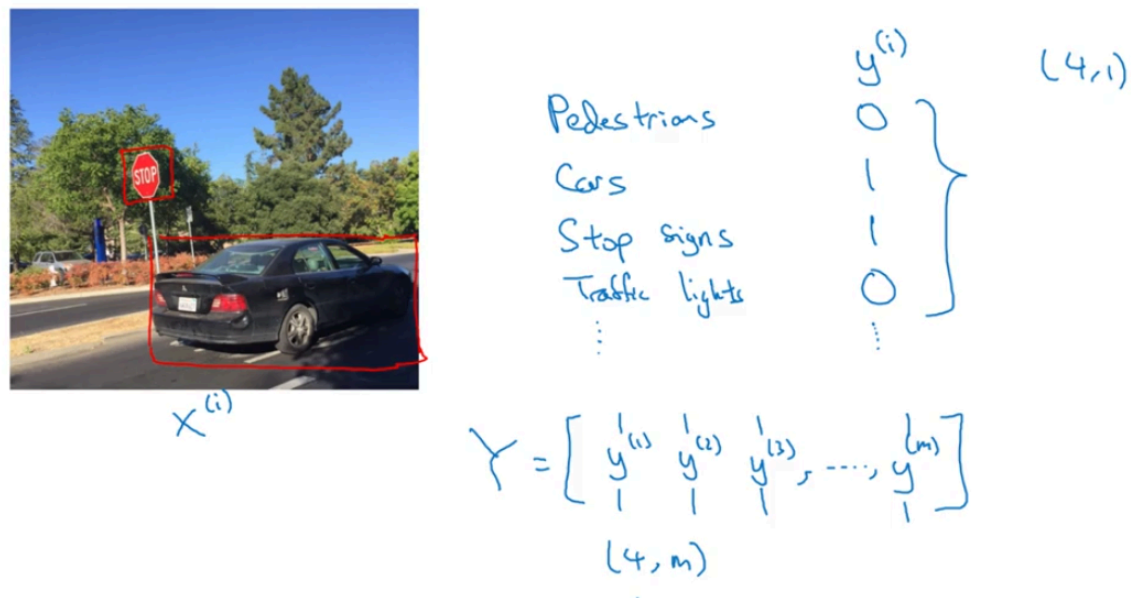
Track from $A \rightarrow B$

- Task A and B have the same input x .
- You have a lot more data for Task A than Task B.

- Low level features from A could be helpful for learning B.

Multi-task Learning

Simplified autonomous driving example



Here's a **simple, easy-to-understand note version**:

Multi-Task Learning (Main Idea)

- **Transfer learning:**
 - Learn task A → then apply to task B (sequential)
- **Multi-task learning:**
 - Learn **multiple tasks at the same time** (simultaneous)

Example: Self-Driving Car

A self-driving car needs to detect many things:

- pedestrians
- cars
- stop signs
- traffic lights

How Labels Work

- Instead of 1 label, we have **multiple labels**

Example for one image:

- No pedestrian → 0
- Car → 1
- Stop sign → 1
- No traffic light → 0

👉 So label becomes a vector:

- **y = [0, 1, 1, 0]**

Dataset Structure

- Each example has:
 - Input: image
 - Output: **vector of multiple labels**

- If there are 4 tasks:
→ y is a **(4 × 1) vector**
- Whole dataset:
→ Y becomes a **4 × m matrix** (m = number of examples)

Model Training

- Train **one neural network**
- Output layer predicts **multiple values at once**
- Each output corresponds to **one task**

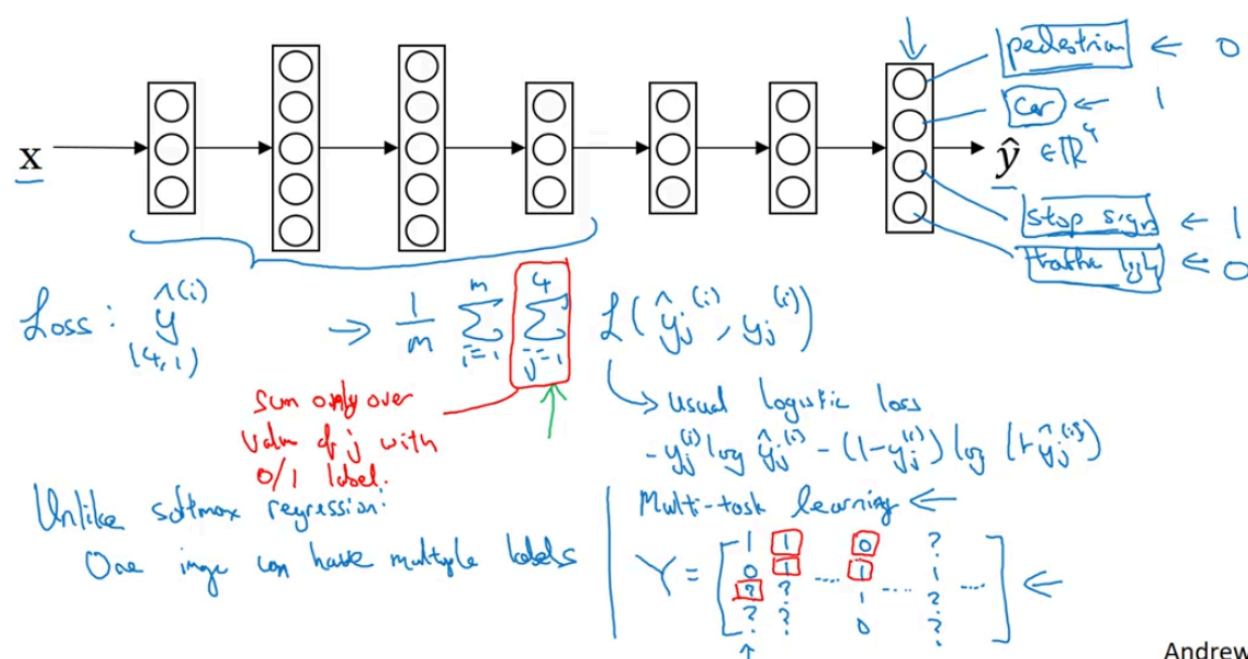
Key Idea

- One model learns:
→ multiple related tasks
→ at the same time
- Tasks can **help each other improve**

Takeaway

- Multi-task learning = **learn many things together**
- Useful when tasks are **related (e.g., object detection)**

Neural network architecture



Here's a **clear, simple note version** you can use:

Multi-Task Learning (Model Output)

- Input: image x
- Output: $\hat{y}$ (**4 values**)

Each output node predicts:

1. Pedestrian
2. Car
3. Stop sign
4. Traffic light

👉 Example output: $\hat{y} = [0, 1, 1, 0]$

Loss Function

- Use **logistic loss for each task**
- Total loss = sum of all tasks

👉 For each training example:

- Compute loss for:
 - pedestrian
 - car
 - stop sign
 - traffic light
 - Then **sum them together**
-

Key Difference from Softmax

- **Softmax (multi-class):**
 - Only **one label per image**
- **Multi-task (multi-label):**
 - **Multiple labels per image**

👉 Example:

- One image can have:
 - car ✓
 - stop sign ✓
 - pedestrian ✗
-

Why Multi-Task Learning Works

- One network learns **multiple tasks together**
- Early layers share features (edges, shapes)

👉 Better than:

- Training **4 separate models**
-

Handling Missing Labels

- Some data may have **missing labels (?)**

Example:

- Know: pedestrian = 1, car = 0
- Unknown: stop sign, traffic light

👉 Solution:

- Only compute loss for **known labels**
 - Ignore missing ones
-

Key Idea

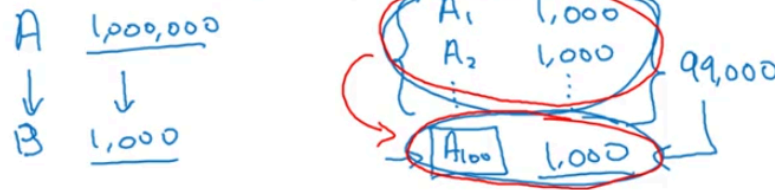
- Train **one model** → **solve many tasks**
 - Share knowledge across tasks
 - Works even with **partially labeled data**
-

Takeaway

- Multi-task learning:
 - Handles **multiple labels per input**
 - Shares features → **better performance**
 - Can train even with **incomplete labels**

When multi-task learning makes sense

- Training on a set of tasks that could benefit from having shared lower-level features.
- Usually: Amount of data you have for each task is quite similar.



- Can train a big enough neural network to do well on all the tasks.

Here's a **simple, structured note version**:

When Multi-Task Learning Works Well

It usually works when **3 conditions** are satisfied:

1. Tasks Share Features

- Tasks should be **related**
- They can share **low-level features** (edges, shapes, patterns)

👉 Example:

- Detecting cars, pedestrians, traffic lights
→ all are **road-related objects**

2. Enough Data Across Tasks

- Each task may have limited data
- But **combined data from all tasks is large**

👉 Idea:

- 1 task = 1,000 examples
- 100 tasks → total = 100,000 examples

→ Other tasks help improve each task

✓ Key insight:

- For one task to benefit
→ other tasks must provide **a lot more data**

3. Model is Big Enough

- Neural network must be **large enough**
- To handle all tasks at the same time

👉 If model is too small:

- Multi-task learning can **hurt performance**

👉 If model is big enough:

- Usually **improves or maintains performance**
-

Multi-Task vs Separate Models

- Option 1: Train **1 model for all tasks**
- Option 2: Train **separate models for each task**

👉 Multi-task is better when:

- Tasks share features
 - Model capacity is sufficient
-

In Practice

- **Transfer learning is used more often**
- Multi-task learning is **less common**

👉 Why?

- Hard to find many related tasks
 - Hard to train one model for everything
-

Common Use Case

- **Computer vision (object detection)**
 - One model detects many objects at once
-

Final Takeaway

- Multi-task learning:
 - 1 model, many tasks
 - can improve performance
- But:
 - Requires related tasks + enough data + big model
- In reality:
 - **Transfer learning is more widely used**